In [1]:
```
import pandas as pd
import numpy as np

import matplotlib.pyplot as plt
import seaborn as sns

from sklearn.model_selection import train_test_split
from sklearn.model_selection import cross_val_score

from sklearn.preprocessing import LabelEncoder

from sklearn.ensemble import RandomForestClassifier

from sklearn.metrics import (
    accuracy_score,
    classification_report,
    confusion_matrix
)

import warnings
warnings.filterwarnings("ignore")
```

In [2]:
```
import pandas as pd

df = pd.read_csv("customer_booking.csv", encoding="latin1")

df.head()
```

Out[2]:

	num_passengers	sales_channel	trip_type	purchase_lead	length_of_stay	flight_hour
0	2	Internet	RoundTrip	262	19	7
1	1	Internet	RoundTrip	112	20	3
2	2	Internet	RoundTrip	243	22	17
3	1	Internet	RoundTrip	96	31	4
4	2	Internet	RoundTrip	68	22	15

In [3]:
```
print(df.shape) # dimensions of the dataset

df.info() # information about the dataset

df.describe() # it shows the statistical summary of the dataset
```

```
(50000, 14)
<class 'pandas.DataFrame'>
RangeIndex: 50000 entries, 0 to 49999
Data columns (total 14 columns):
 #   Column                 Non-Null Count  Dtype  
---  ------                 --------------  -----  
 0   num_passengers         50000 non-null  int64  
 1   sales_channel          50000 non-null  str    
 2   trip_type              50000 non-null  str    
 3   purchase_lead          50000 non-null  int64  
 4   length_of_stay         50000 non-null  int64  
 5   flight_hour            50000 non-null  int64  
 6   flight_day             50000 non-null  str    
 7   route                  50000 non-null  str    
 8   booking_origin         50000 non-null  str    
 9   wants_extra_baggage    50000 non-null  int64  
 10  wants_preferred_seat   50000 non-null  int64  
 11  wants_in_flight_meals  50000 non-null  int64  
 12  flight_duration        50000 non-null  float64
 13  booking_complete       50000 non-null  int64  
dtypes: float64(1), int64(8), str(5)
memory usage: 7.0 MB
```

Out[3]:

	num_passengers	purchase_lead	length_of_stay	flight_hour	wants_extra_baggag
count	50000.000000	50000.000000	50000.00000	50000.00000	50000.00000
mean	1.591240	84.940480	23.04456	9.06634	0.66878
std	1.020165	90.451378	33.88767	5.41266	0.47065
min	1.000000	0.000000	0.00000	0.00000	0.00000
25%	1.000000	21.000000	5.00000	5.00000	0.00000
50%	1.000000	51.000000	17.00000	9.00000	1.00000
75%	2.000000	115.000000	28.00000	13.00000	1.00000
max	9.000000	867.000000	778.00000	23.00000	1.00000

In [4]:
```
df.isnull().sum()
```

Out[4]:
```
num_passengers          0
sales_channel           0
trip_type               0
purchase_lead           0
length_of_stay          0
flight_hour             0
flight_day              0
route                   0
booking_origin          0
wants_extra_baggage     0
wants_preferred_seat    0
wants_in_flight_meals   0
flight_duration         0
booking_complete        0
dtype: int64
```

In [5]:
```
df.duplicated().sum() # it shows the number of duplicated rows in the dataset
```

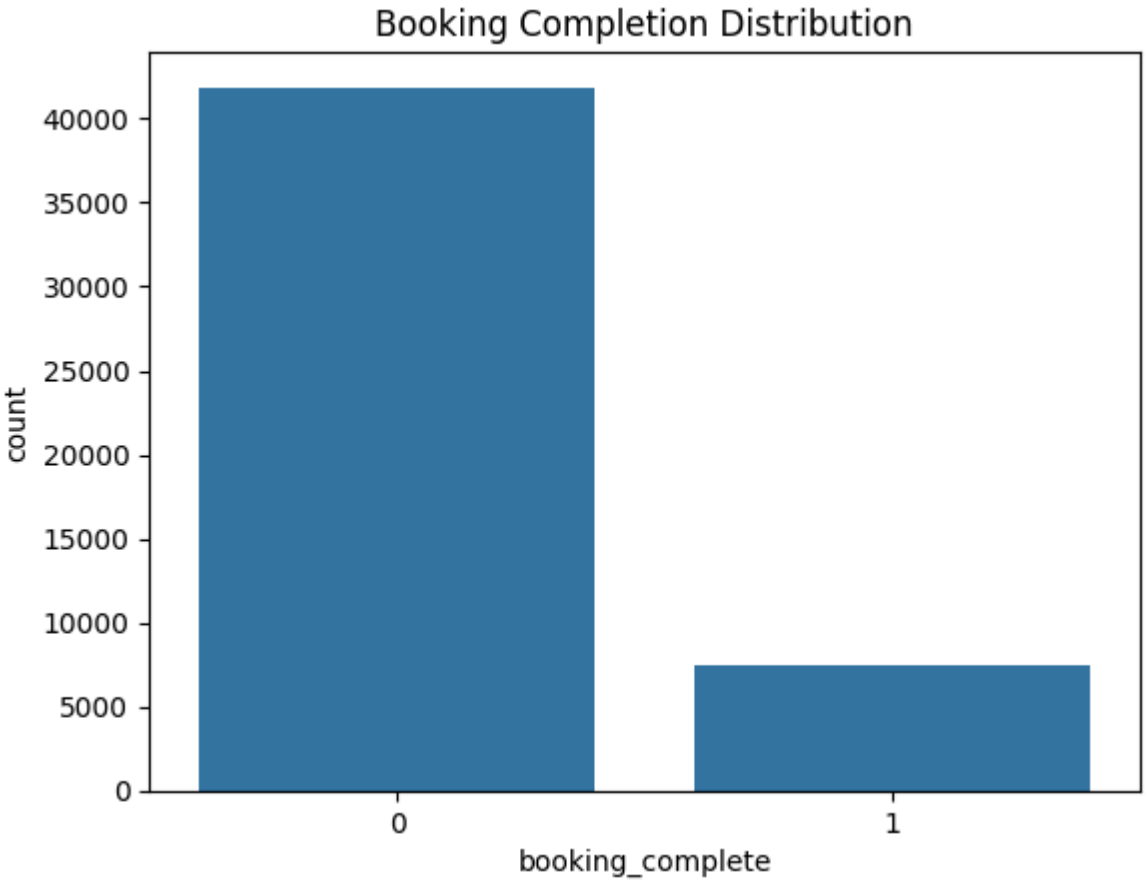
Out[5]:
```
np.int64(719)
```

In [6]:
```
df = df.drop_duplicates() # it removes the duplicated rows from the dataset
```

In [7]:
```
# target variable distribution
df["booking_complete"].value_counts()
```

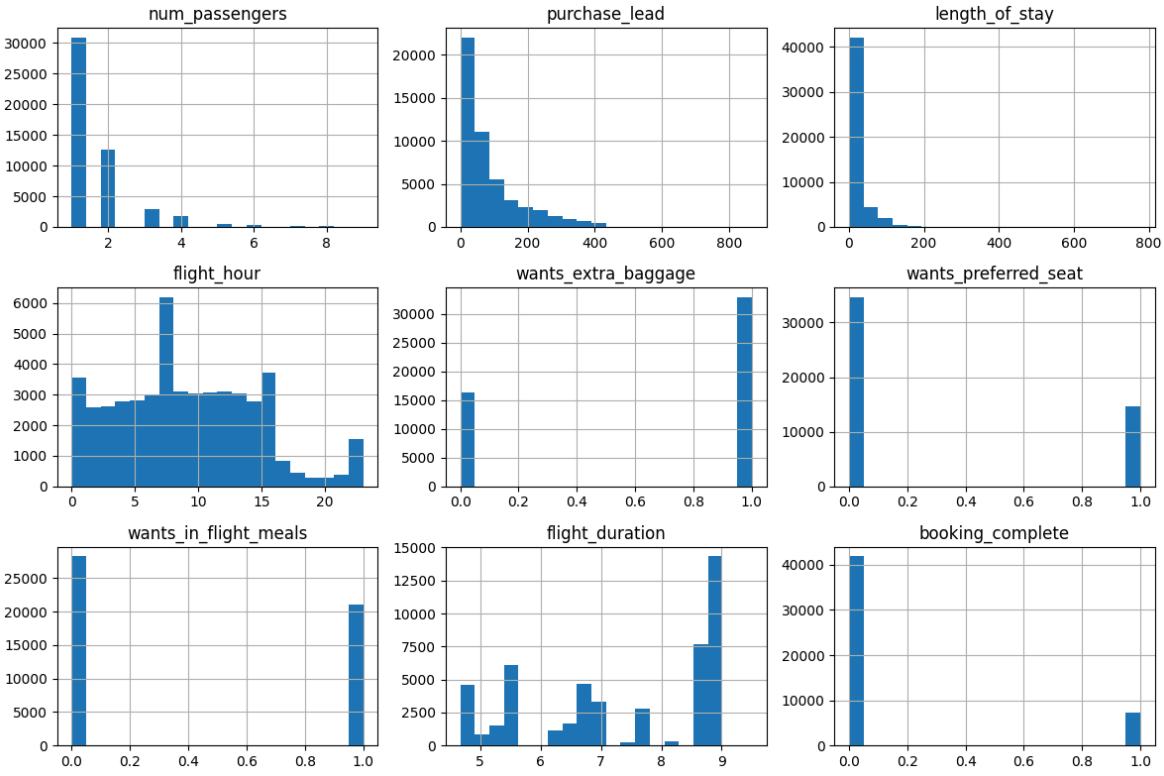
Out[7]:
```
booking_complete
0    41890
1     7391
Name: count, dtype: int64
```

In [8]:
```
# target variable distribution

sns.countplot(x="booking_complete", data=df)

plt.title("Booking Completion Distribution")

plt.show()
```

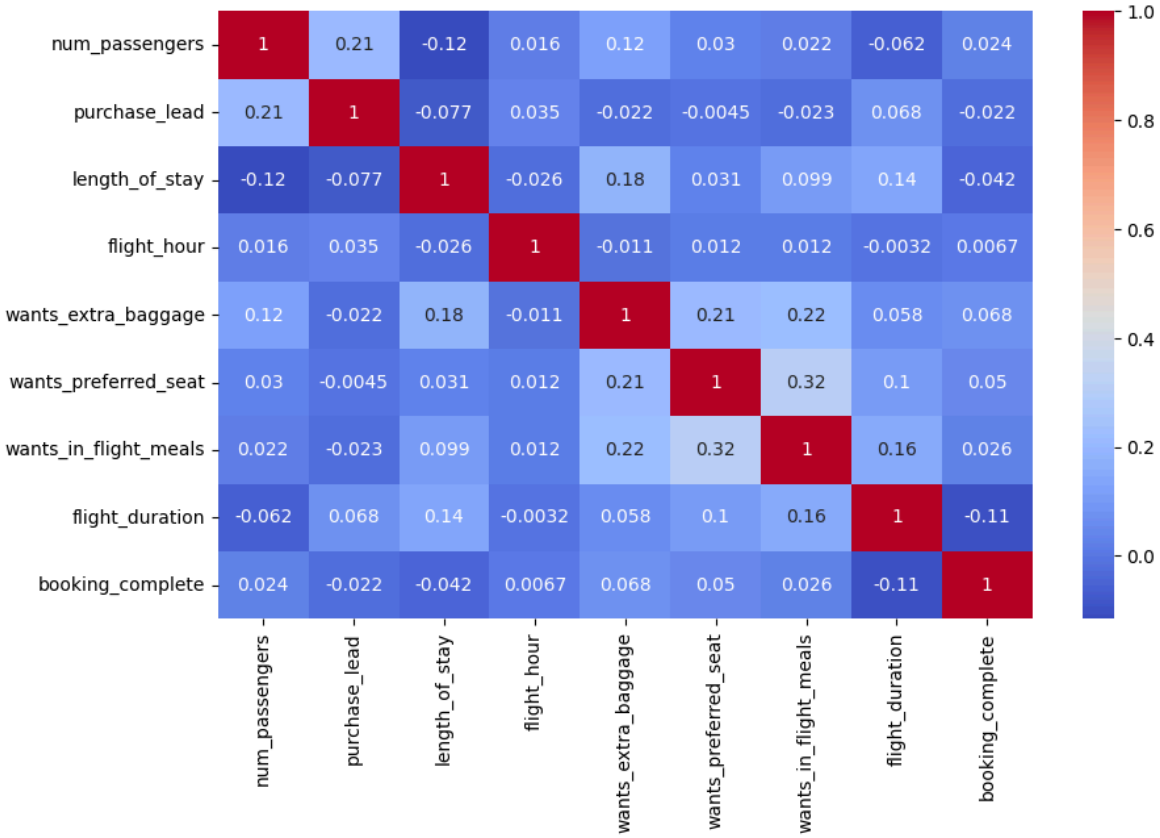


In [9]:
```
# numerical features distribution

num_cols = df.select_dtypes(include=np.number).columns

df[num_cols].hist(
    figsize=(12,8),
    bins=20
)

plt.tight_layout()
```

```
plt.show()
```



In [10]:
```
# correlation heatmap
plt.figure(figsize=(10,6))

sns.heatmap(
    df.corr(numeric_only=True),
    annot=True,
    cmap="coolwarm"
)

plt.show()
```



In [11]:
```
# categorical columns

cat_cols = df.select_dtypes(include="object").columns

cat_cols
```

Out[11]:
```
Index(['sales_channel', 'trip_type', 'flight_day', 'route', 'booking_origin'],
dtype='str')
```

In [12]:
```
# label encoding for categorical variables
le = LabelEncoder()

for col in cat_cols:
    df[col] = le.fit_transform(df[col])
```

In [13]:
```
X = df.drop("booking_complete", axis=1)

y = df["booking_complete"]
```

In [14]:
```
# train-test split

X_train, X_test, y_train, y_test = train_test_split(
    X,
    y,
    test_size=0.2,
    random_state=42
)
```

In [15]:
```
# Random Forest Classifier

rf = RandomForestClassifier(
    n_estimators=200,
    random_state=42
```

```
)

rf.fit(X_train, y_train)
```

Out[15]:

RandomForestClassifier

▶ Parameters

In [16]:
```
y_pred = rf.predict(X_test)
```

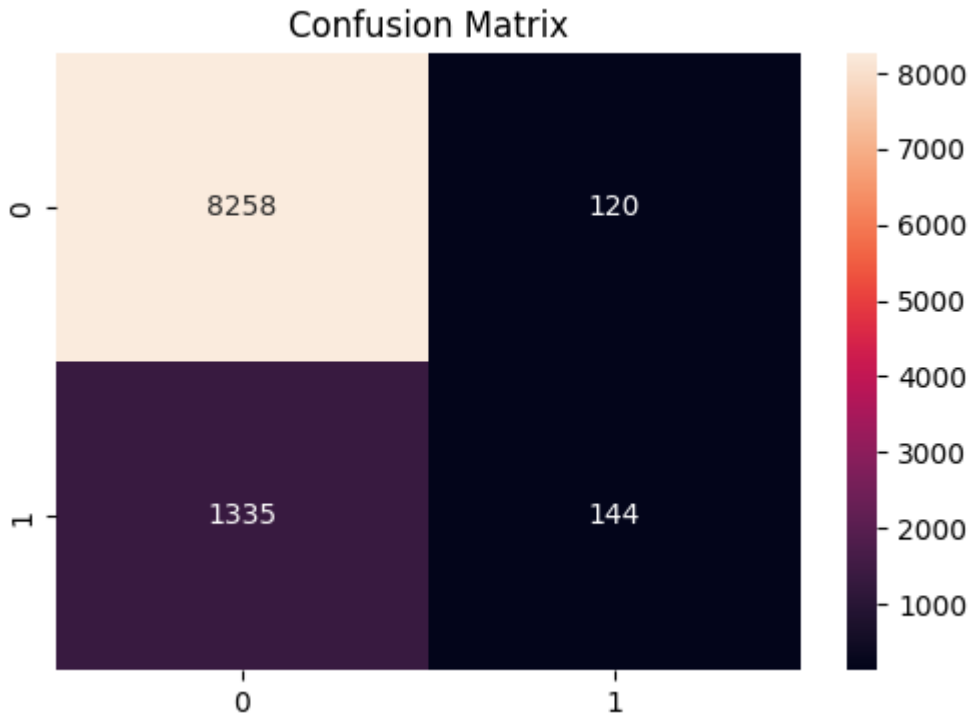
In [17]:
```
# Accuracy Score

accuracy = accuracy_score(
    y_test,
    y_pred
)

print("Accuracy:", accuracy)
```

```
Accuracy: 0.8523891650603632
```

In [18]:
```
print(
    classification_report(
        y_test,
        y_pred
    )
)
```

```
              precision    recall  f1-score   support

           0       0.86      0.99      0.92      8378
           1       0.55      0.10      0.17      1479

    accuracy                           0.85      9857
   macro avg       0.70      0.54      0.54      9857
weighted avg       0.81      0.85      0.81      9857
```

In [19]:
```
# Confusion Matrix

cm = confusion_matrix(
    y_test,
    y_pred
)

plt.figure(figsize=(6,4))

sns.heatmap(
    cm,
    annot=True,
    fmt="d"
)

plt.title("Confusion Matrix")

plt.show()
```



In [20]:
```
# Cross-validation scores

cv_scores = cross_val_score(
    rf,
    X,
    y,
    cv=5
)

print("CV Scores")

print(cv_scores)

print("Average CV Score")

print(cv_scores.mean())
```

```
CV Scores
[0.8504616  0.47554789 0.25355114 0.36576705 0.5367289 ]
Average CV Score
0.4964113136850468
```

In [21]:
```
# feature importance

importance = rf.feature_importances_

feature_importance = pd.DataFrame(
    {
        "Feature": X.columns,
        "Importance": importance
    }
)

feature_importance = feature_importance.sort_values(
    by="Importance",
    ascending=False
)
```

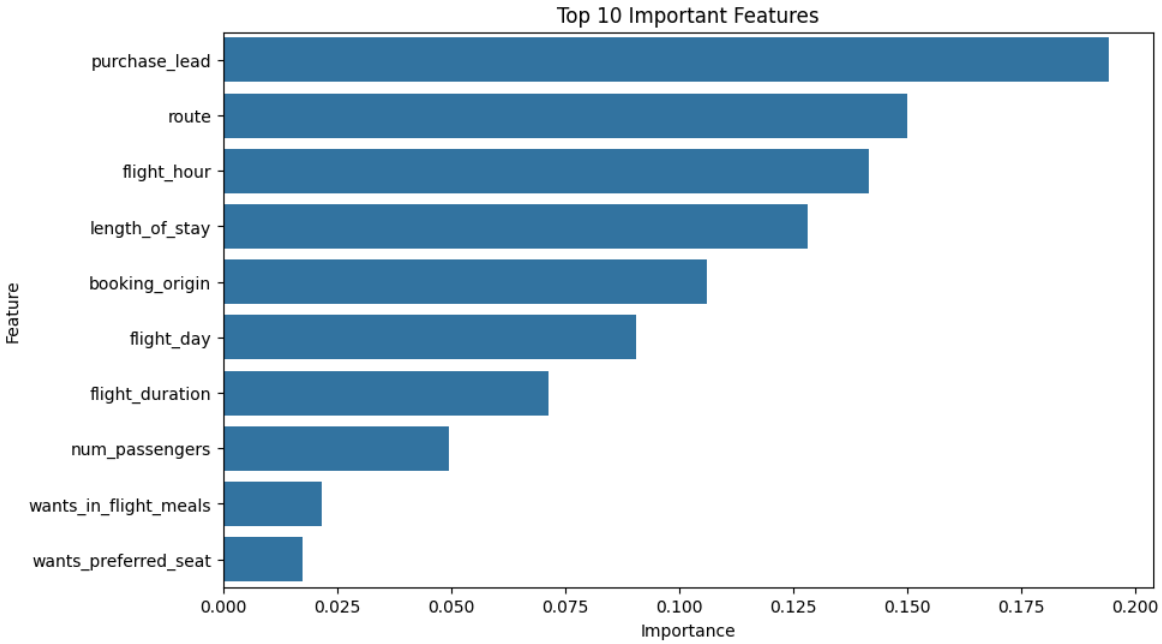
```
feature_importance.head(10)
```

Out[21]:

	Feature	Importance
3	purchase_lead	0.194266
7	route	0.150144
5	flight_hour	0.141749
4	length_of_stay	0.128351
8	booking_origin	0.106006
6	flight_day	0.090568
12	flight_duration	0.071262
0	num_passengers	0.049608
11	wants_in_flight_meals	0.021542
10	wants_preferred_seat	0.017516

In [22]:

```
# display top 10 important features

plt.figure(figsize=(10,6))

sns.barplot(
    data=feature_importance.head(10),
    x="Importance",
    y="Feature"
)

plt.title(
    "Top 10 Important Features"
)

plt.show()
```



In [23]:
```
# display top 10 important features

print(
feature_importance.head(10)
)
```

	Feature	Importance
3	purchase_lead	0.194266
7	route	0.150144
5	flight_hour	0.141749
4	length_of_stay	0.128351
8	booking_origin	0.106006
6	flight_day	0.090568
12	flight_duration	0.071262
0	num_passengers	0.049608
11	wants_in_flight_meals	0.021542
10	wants_preferred_seat	0.017516

In [24]:
```
# 5 customers prediction

for i in range(5):
    customer = X_test.iloc[[i]]

    pred = rf.predict(customer)[0]
    prob = rf.predict_proba(customer)[0]

    print(f"\nCustomer {i+1}")

    if pred == 1:
        print(f"Booking Complete ({prob[1]*100:.2f}%)")
    else:
        print(f"Booking Not Complete ({prob[0]*100:.2f}%)")
```

```
Customer 1
Booking Not Complete (75.00%)

Customer 2
Booking Not Complete (76.50%)

Customer 3
Booking Not Complete (99.50%)

Customer 4
Booking Not Complete (82.50%)

Customer 5
Booking Not Complete (85.50%)
```

In [25]:
```
# Final Business Insights

print("="*60)
print("BUSINESS INSIGHTS")
print("="*60)

top_features = feature_importance.head(5)

print("\nTop 5 Most Important Features:")
print(top_features)

print("\nKey Findings:")
```

```
print("1. The model successfully identifies factors influencing booking completi
print("2. Customers purchasing additional services (baggage, meals, seats) are m
print("3. Purchase lead time plays an important role in predicting customer beha
print("4. Flight-related factors such as route and duration contribute to bookin
print("5. The dataset is imbalanced, with significantly fewer completed bookings

print("\nBusiness Recommendations:")
print("• Target high-intent customers with personalized marketing campaigns.")
print("• Promote ancillary services such as baggage and meal packages.")
print("• Focus marketing efforts on customer segments with higher booking comple
print("• Use predictive analytics to improve conversion rates and customer acqui
```

```
============================================================
BUSINESS INSIGHTS
============================================================

Top 5 Most Important Features:
          Feature  Importance
3   purchase_lead    0.194266
7           route    0.150144
5     flight_hour    0.141749
4  length_of_stay    0.128351
8  booking_origin    0.106006

Key Findings:
1. The model successfully identifies factors influencing booking completion.
2. Customers purchasing additional services (baggage, meals, seats) are more like
ly to complete bookings.
3. Purchase lead time plays an important role in predicting customer behavior.
4. Flight-related factors such as route and duration contribute to booking decisi
ons.
5. The dataset is imbalanced, with significantly fewer completed bookings than in
complete bookings.

Business Recommendations:
• Target high-intent customers with personalized marketing campaigns.
• Promote ancillary services such as baggage and meal packages.
• Focus marketing efforts on customer segments with higher booking completion pro
bability.
• Use predictive analytics to improve conversion rates and customer acquisition.
```